

The Discrete Logarithm Problem

Maria Eichlseder

Some parts based on slides by Elisabeth Oswald and Mario Lamberger

Cryptanalysis – ST 2023

Outline

The Discrete Logarithm Problem (DLP)

General DLP Algorithms

- Naïve methods
- Babystep-Giantstep
- Pollard- ρ

Special DLP Algorithms for Special Groups

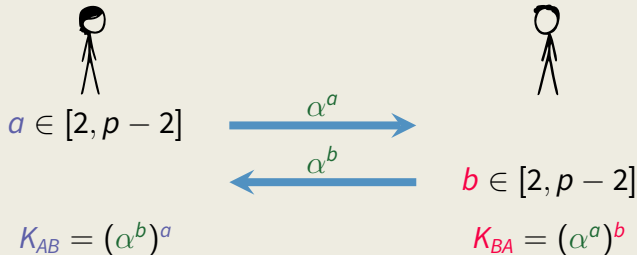
- Index-Calculus

The Discrete Logarithm Problem (DLP)



Motivation: Diffie–Hellman key agreement (DH, 1976)

Diffie–Hellman key agreement



where p is a large prime and α a generator of $\mathbb{Z}_p^*$ (both public)

Security = **Diffie–Hellman Problem**: How to find $K_{AB} = \alpha^{a \cdot b}$ from α^a and α^b ?

Best known solution: find a from α^a , or b from α^b (= solve DLP in $\mathbb{Z}_p^*$)

Reminder: Cyclic groups

Cyclic Groups

A group G is **cyclic** if there is a **generator** element $g \in G$ such that every $a \in G$ is a power of g , i.e., for some integer t ,

$$a = g^t = \underbrace{g * g * \cdots * g}_{t \text{ times}}$$

Group Order & Element Order

The **order of the group** G is the number of elements in G .

The **order of an element** $a \in G$ is the smallest $t \in \mathbb{N}$ such that $a^t = 1$.

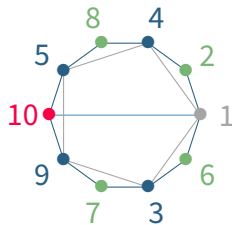
$$t \mid |G|$$

Cyclic groups: Example $(\mathbb{Z}_{11}^*, \cdot)$

Consider $\mathbb{Z}_{11}^* = \{1, 2, \dots, 10\}$ (order 10, since 11 is prime).

The subgroups of $(\mathbb{Z}_{11}^*, \cdot)$ are:

Subgroup	Generators	Order
$\{1\}$	1	1
$\{1, 10\}$	10	2
$\{1, 3, 4, 5, 9\}$	3, 4, 5, 9	5
$\{1, 2, \dots, 10\}$	2, 6, 7, 8	10



$\mathbb{Z}_{11}^*$ is cyclic, and the elements 2, 6, 7, 8 are generators.

Subgroups and Generators: Some Observations

- Order of an element a = order of subgroup generated by a .
This order always divides the group order.

❗ The order of a^k is $\text{ord}(a^k) = \frac{\text{ord}(a)}{\gcd(\text{ord}(a), k)}$.

- ❗ $\mathbb{Z}_p^*$ contains exactly $\varphi(d)$ elements of order d for each divisor d of $p - 1$.

For $d \geq 5$, $\varphi(d) \geq \frac{d}{6 \log \log d}$.

That's good news: Most elements have a high order.

Discrete Logarithm Problem (DLP)

Generalized Discrete Logarithm Problem

Given a finite cyclic group G of order n with

- a generator $g \in G$ of the group,
- a group element $y \in G$,

find

- an integer $x \in \{0, \dots, n - 1\}$ such that $g^x = y$.

The DLP is believed to be a hard problem.

DLP difficulty is independent of the generator

Suppose we have two generators α and β of $\mathbb{Z}_p^*$ and two DLPs:

$$\alpha^x \equiv y \pmod{p} \qquad \beta^z \equiv y \pmod{p}$$

β is also generated by α as $\beta = \alpha^w$ (and vice versa), so

$$y \equiv \alpha^x \equiv (\alpha^w)^z \pmod{p}$$

$$\Rightarrow x \equiv wz \pmod{p-1}$$

$$\Rightarrow \log_\alpha y \equiv x \equiv wz \equiv \log_\alpha \beta \log_\beta y \pmod{p-1}.$$

$\Rightarrow$ any algorithm that efficiently computes $\log_\alpha$ can be used to compute $\log_\beta$ by

$$\log_\beta y \equiv \frac{\log_\alpha y}{\log_\alpha \beta} \pmod{p-1}.$$

DLP difficulty highly depends on the group representation

- Any two cyclic groups of the same order are isomorphic
(have the same group structure)
- But having an efficient algorithm to solve the DLP in one group does not imply an efficient algorithm for the other group!
- ❗ DLP in $(\mathbb{Z}_n, +)$ is **trivial**: need to solve $g + g + \dots + g = g \cdot x \equiv y \pmod{n}$.
(That's what the extended Euclidean algorithm solves!)
- ➡ Choosing the right group is essential for cryptosystems like Diffie-Hellman!
 - ✓ Multiplicative group $\mathbb{Z}_p^*$ for a (safe) prime p
 - ✓ Elliptic-curve group

General DLP Algorithms



Solving the DLP in any group

Exhaustive Search

🎯 **Goal:** given $y = g^x$ in group G of order n , find x .

💡 **Idea:** try if $g^1, g^2, g^3, \dots$ matches y

⌚ **Complexity:** depends on order of g , at most $\mathcal{O}(n)$ (exponential!).

For example, for a 256-bit modulus p , it takes roughly 2^{256} tries.

Exhaustive Search – Build an Index Table

💡 Can trade time for memory (and precomputation):

- **Precompute index table** with all powers of a generator g
(sort by result)
- **Query:** compute $\log_g$ with table lookup
(use log rules for other generators)

Example: Index table for 2 in $\mathbb{Z}_{11}^*$:

x	0	1	2	3	4	5	6	7	8	9
$2^x \pmod{11}$	1	2	4	8	5	10	9	7	3	6

🕒 Table size $\mathcal{O}(n)$, precomputation time $\mathcal{O}(n)$, query time $\mathcal{O}(1)$

Babystep-Giantstep method (BSGS)

A more balanced time-memory tradeoff by Shanks:

🎯 **Goal:** given $y = g^x$ in group G of order $\leq n$, find x .

💡 **Idea:** Write x as

$$x = i \cdot m + j \quad \text{with } m = \lceil \sqrt{n} \rceil \text{ and } i, j < m$$

$$\Rightarrow y = g^{im} \cdot g^j$$

$$\Rightarrow y \cdot g^{-mi} = g^j.$$

- **Baby steps:** Precompute all g^j and store in index table.
- **Giant steps:** Compute $y \cdot g^{-mi}$ and look for match in table.

If match occurs, recover x from i and j .

🕒 **Complexity:** table size $\mathcal{O}(\sqrt{n})$, time $\mathcal{O}(\sqrt{n})$

Babystep-Giantstep method: Example

Goal: Solve $22 \equiv 2^x \pmod{37}$.

37 is prime $\rightarrow$ group order is $37 - 1 = 36$. Write

$$x = i\sqrt{36} + j = 6i + j.$$

$$n = 36$$

$$5 = \sqrt{n} - 1 =$$

Baby steps:

j	0	1	2	3	4	5
$2^j \pmod{37}$	1	2	4	8	16	32

Giant steps:

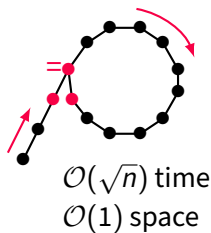
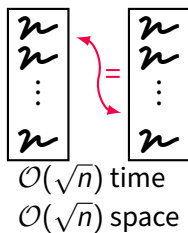
i	5	3	4	1	0	2
$y \cdot 2^{-mi} \pmod{37}$	2	15	17	20	22	35

$$y \cdot (2^{-m})^i$$

2 is in both tables: $i = 5$ and $j = 1$, so $x = i \cdot m + j = 5 \cdot 6 + 1 = 31$.

Pollard- ρ method: Memoryless Collisions

- 💡 Idea: BSGS essentially finds collisions between lists.
- ➡ We can reuse some ideas from memoryless hash collision search



Pollard- ρ method: Application to DLP

We want to solve $y = g^x \pmod{p}$ by finding a solution for

$$r_j = y^{a_j} g^{b_j} = y^{a_k} g^{b_k} = r_k \pmod{p}$$

If we find $r_j = r_k$, then $x \cdot (a_j - a_k) \equiv (b_k - b_j) \pmod{p-1}$ (why?).

We start with $r_0 = 1$ ($a_0 = b_0 = 0$) and update $r_i = y^{a_i} g^{b_i} \pmod{p} \rightarrow r_{i+1}$:

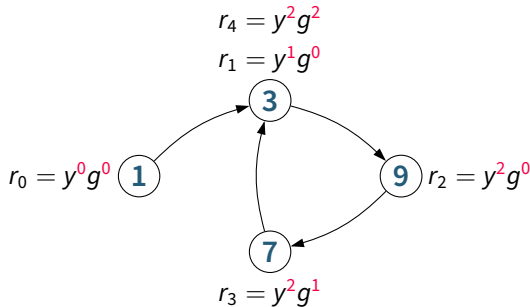
$$r_{i+1} = \begin{cases} y \cdot r_i & \pmod{p} \\ r_i^2 & \pmod{p}, \\ g \cdot r_i & \pmod{p} \end{cases}, \quad (a_{i+1}, b_{i+1}) = \begin{cases} (a_i + 1, b_i) & \pmod{p-1} & 0 < r_i < \frac{p}{3} \\ (2a_i, 2b_i) & \pmod{p-1} & \frac{p}{3} < r_i < \frac{2p}{3} \\ (a_i, b_i + 1) & \pmod{p-1} & \frac{2p}{3} < r_i < p \end{cases}$$

Now apply a cycle-finding method like Floyd's algorithm to this update function.

Pollard- ρ method: Example

Solve: $3 \equiv 2^x \pmod{11}$, so $y = 3, g = 2$

rule	graph node
init	$r_0 = y^0 g^0 \equiv 1$
$(a+1, b)$	$r_1 = y^1 g^0 = y \cdot r_0 \equiv 3$
$(a+1, b)$	$r_2 = y^2 g^0 = y \cdot r_1 \equiv 9$
$(a, b+1)$	$r_3 = y^2 g^1 = g \cdot r_2 \equiv 7$
$(a, b+1)$	$r_4 = y^2 g^2 = g \cdot r_3 \equiv 3$



Found $y^1 g^0 \equiv y^2 g^2 \Rightarrow g^{x \cdot (1-2)} = g^{2-0} \pmod{11} \Rightarrow x \equiv \frac{2-0}{1-2} \equiv -2 \equiv 8 \pmod{10}$

The actual steps of Floyd's memoryless cycle-finding (Cryptography L5) on this graph would be:

step i	turtle r_i	hare r_{2i}
0	$r_0 = 1$	$r_0 = 1$
1	$r_1 = 3$	$r_2 = 9$
2	$r_2 = 9$	$r_4 = 3$
3	$r_3 = 7$	$r_6 = 7$

$\Rightarrow$ cycle length
 $\lambda = i = 3$

step i	turtle $r_{i+\lambda}$	restart r_i
0	$r_3 = 7$	$r_0 = 1$
1	$r_4 = 3$	$r_1 = 3$

tail length
 $\mu = i = 1$
collision
 $r_{i+\lambda} = r_i$
 $r_4 = r_1$

Generic methods: Summary

 Generic DLP attacks (BSGS, Pollard- ρ) have complexity $\mathcal{O}(\sqrt{n})$

 n -bit security requires group order $\geq 2^{2n}$

(e.g., at least 256-bit prime modulus for 128-bit security)

Special DLP Algorithms for Special Groups



Index-Calculus Algorithm

💡 Similar approach to modern factoring algorithms (sieving):

- 1 **Configure**: Choose a factor base $\mathcal{B} = \{p_1, p_2, \dots, p_B\}$
- 2 **Collect relations**: Determine DL of primes in $\mathcal{B}$
- 3 **Combine**: Compute DL of y based on DL of primes in $\mathcal{B}$

Tasks 1, 2 are precomputation, Task 3 is repeated for each query.

📌 Complexity similar to sieving factorization: $L_n[\frac{1}{2}, \sqrt{2}]$ or faster

Index-Calculus Task 2: Determine DL of primes in $\mathcal{B}$

- For random t_i , try to factor g^{t_i} over $\mathcal{B}$ to get many relations

$$g^{t_i} \equiv p_1^{a_{1i}} \cdot p_2^{a_{2i}} \cdot \dots \cdot p_B^{a_{Bi}} \pmod{p}$$

- Rewrite these relations:

$$t_i \equiv a_{1i} \log_g p_1 + a_{2i} \log_g p_2 + \dots + a_{Bi} \log_g p_B \pmod{p-1}$$

- Solve this system of equations to determine the $\log_g p_i$.

(if more equations than unknowns $\rightarrow$ unique solution mod $p-1$)

Index-Calculus Task 3: Compute DL of y

We want to solve $y \equiv g^x \pmod{p}$. Repeat until successful:

- Choose a random number s ($1 \leq s \leq p - 2$) and compute

$$y \cdot g^s \pmod{p}.$$

- Try to factor this number over the factor base. If successful, we have

$$y \cdot g^s \equiv p_1^{c_1} \cdot p_2^{c_2} \cdot \dots \cdot p_B^{c_B} \pmod{p}.$$

- Applying $\log_g$ on both sides allows us to recover $x = \log_g y$:

$$\log_g y + s \equiv c_1 \log_g p_1 + c_2 \log_g p_2 + \dots + c_B \log_g p_B \pmod{p - 1}.$$

Index-Calculus: Example I

Solve $37 \equiv 2^x \pmod{131}$ using the factor base $\mathcal{B} = \{2, 3, 5, 7\}$:

Task 2: Choose some random numbers $t_i = \{1, 8, 12, 14, 34\}$, factor g^{t_i} over $\mathcal{B}$:

$$2^1 \equiv 2 = 2 \pmod{131}$$

$$2^8 \equiv 125 = 5^3 \pmod{131}$$

$$2^{12} \equiv 35 = 5 \cdot 7 \pmod{131}$$

$$2^{14} \equiv 9 = 3^2 \pmod{131}$$

$$2^{34} \equiv 75 = 3 \cdot 5^2 \pmod{131}$$



$$1 \equiv \log_2 2 \pmod{130}$$

$$8 \equiv 3 \log_2 5 \pmod{130}$$

$$12 \equiv \log_2 5 + \log_2 7 \pmod{130}$$

$$14 \equiv 2 \log_2 3 \pmod{130}$$

$$34 \equiv \log_2 3 + 2 \log_2 5 \pmod{130}$$

Solve system to get $\log_2$ of primes in $\mathcal{B}$:

$$\log_2 2 \equiv 1 \pmod{130}$$

$$\log_2 5 \equiv 8 \cdot 3^{-1} \equiv 46 \pmod{130}$$

$$\log_2 7 \equiv 12 - 46 \equiv 96 \pmod{130}$$

$$\log_2 3 \equiv 34 - 2 \cdot 46 \equiv 72 \pmod{130}$$

Note: $14 \equiv 2 \log_2 3$ has 2 solutions, 7 and 72 (since $\gcd(2, 130) \neq 1$), so we collect an extra equation for a unique solution.

Index-Calculus: Example II

Task 3: Attempt to compute the DL of 37:

Try random s , compute $y \cdot g^s \pmod{p}$ and hope it factors over $\mathcal{B}$:

$$s = 2: \quad 37 \cdot 2^2 \equiv 17 = 17 \pmod{131}$$

$$s = 3: \quad 37 \cdot 2^3 \equiv 34 = 2 \cdot 17 \pmod{131}$$

$$\vdots \qquad \qquad \qquad \vdots$$

$$s = 6: \quad 37 \cdot 2^6 \equiv 10 = 2 \cdot 5 \pmod{131}$$

Success for $s = 6$, we get

$$\log_2 37 = -s + \sum c_i \log_g p_i = -6 + 1 \cdot 1 + 1 \cdot 46 \equiv 41 \pmod{130}$$

Advanced methods: Summary

- **Pohlig-Hellman** (not covered today):
 - Reduces DLP in large group to smaller subgroups
 - Efficient if group order has only small prime factors
 - Complexity: $\mathcal{O}\left(\sum_i e_i(\log n + \sqrt{p_i})\right)$
- **Index-Calculus:**
 - Most efficient algorithm
 - Requires “prime elements” and “factorization”
 - Not applicable to Elliptic Curves!
- Efficient quantum algorithm also exists (Shor)

Factoring and Discrete Logarithm records

Factoring

using General Number Field Sieve

768-bit modulus (Kleinjung et al. 2009, in 1700 core years)

795-bit modulus (Boudot et al. 2019, in 900 core years)

829-bit modulus (Boudot et al. 2020, in 2700 core years)

Discrete Logarithm

using General Number Field Sieve

768-bit group order (Kleinjung et al. 2017, in 5300 core years; 2 days/challenge)

795-bit group order (Boudot et al. 2019, in 3100 core years)

Elliptic-Curve Discrete Logarithm

using Pollard- ρ

118-bit binary curve (Bernstein et al. 2016, 250 FPGA years)

The complexities correspond, very roughly, to
 $\approx 2^{60}$ crypto operations (118-bit EC) or $\approx 2^{76}$ simple operations (768-bit NFS).

Discrete Logarithm – Summary

- All generic algorithms have a running time of $\mathcal{O}(\sqrt{n})$.
- The best known algorithms to solve the DL are the index-calculus methods. They are based on sieving.
- Hardness of DLP is closely related to that of factoring.
- Security implications:
 - Generic methods: choose group order $\geq 2^{2n}$ for n -bit security
 - Pohlig-Hellman: choose p such that $p - 1$ has large prime factor
 - Index-Calculus: choose higher group order for $\mathbb{Z}_p^*$

Questions you should be able to answer

1. What is the discrete logarithm problem (DLP)? How is its hardness related to the generator of the group, how to the representation of the group?
2. Name two methods for solving the DLP and explain how they work.
3. Explain the Pollard- ρ algorithm. What is its computational complexity?
4. Explain the Index Calculus Algorithm and its relation to modern factoring methods.

Bibliography

- [Adl79] Leonard M. Adleman. **A Subexponential Algorithm for the Discrete Logarithm Problem with Applications to Cryptography**. Foundations of Computer Science – FOCS. IEEE Computer Society, 1979, pp. 55–60. DOI: [10.1109/SFCS.1979.2](https://doi.org/10.1109/SFCS.1979.2).
- [PH78] Stephen C. Pohlig and Martin E. Hellman. **An improved algorithm for computing logarithms over $GF(p)$ and its cryptographic significance**. IEEE Trans. Information Theory 24.1 (1978), pp. 106–110. DOI: [10.1109/TIT.1978.1055817](https://doi.org/10.1109/TIT.1978.1055817).
- [Pol78] John M. Pollard. **Monte Carlo Methods for Index Computation (mod p)**. Mathematics of Computation 32.143 (1978), pp. 918–924. DOI: [10.2307/2006496](https://doi.org/10.2307/2006496).
- [Sha71] Daniel Shanks. **Class number, a theory of factorization, and genera**. 1969 Number Theory Institute. Proceedings of Symposia in Pure Mathematics 20 (1971), pp. 415–440.